

Assignment 4: JVM Code Generation for MiRust

Version: v1.0

1. OBJECTIVES

- Understand how block-structured language features (functions, local variables, structs, loops with break/continue, conditional branches, switch statements, expressions) compile to JVM bytecode.
 - Learn stack-oriented code generation techniques and local variable slot management.
 - Implement structure compilation by generating separate public classes on the JVM.
 - Build a robust compilation and execution testing harness using the Jasmin assembler and JVM.
 - Create a comprehensive suite of unit tests to verify your implementation.
-

2. REQUIRED TASKS TO COMPLETE

- **Task 1: Study the AST Node definitions**
 - Read `src/utils/AST.py` to ensure you understand all the abstract syntax tree representations.
 - **Task 2: Implement CodeGen.py and Emitter.py**
 - Complete the class `CodeGenerator` in `src/codegen/CodeGen.py` (inheriting from `BaseVisitor`).
 - Complete the helper class `Emitter` in `src/codegen/Emitter.py` to generate correct JVM descriptors and Jasmin instructions.
 - Generate a main public class named `MiRustProgram` containing all compiled static functions. The `MiRust` `main` function MUST be translated into the standard Java entry point `public static void main(String[] args)` inside this `MiRustProgram` class.
 - Create a separate JVM class file for each `struct` defined in the program.
 - **Task 3: Write 100 codegen tests**
 - Provide 100 test cases in `test/test-codegen.py` (`test_1` to `test_100`) using the provided `CodeGenerator` helper.
 - Verify all aspects of variable assignments, scopes, loops, struct fields, functions, and control flow.
-

3. PREDEFINED STANDARD FUNCTIONS & INTRINSICS

Your code generator must support the following predefined functions:

3.1. Standard I/O Functions

- `print_i32(val: i32) -> void` / `println_i32(val: i32) -> void`
- `print_f32(val: f32) -> void` / `println_f32(val: f32) -> void`
- `print_bool(val: bool) -> void` / `println_bool(val: bool) -> void`
- `print_string(val: string) -> void` / `println_string(val: string) -> void`
- `println() -> void` (prints a newline)

3.2. Memory Allocation Intrinsic

- `Box_new(val: T) -> Box<T>` (allocates a value on the heap)
 - *Implementation Note:* Students may implement `Box_new` by emitting **inline JVM bytecode** (e.g., allocating a 1-element reference array on the heap via

`newarray` / `anewarray` and storing the argument value) rather than strictly invoking a method in `io.java`. Inlining bytecode is particularly advantageous for user-defined structs as it avoids the need for explicit type casting (`checkcast`).

4. PROJECT STRUCTURE

The repository layout for Assignment 4 must be structured as follows:

```
root/
├── src/
│   ├── grammar/
│   │   ├── MiRust.g4          # Student's ANTLR4 grammar file
│   │   └── (generated files) # Generated by ANTLR (Lexer, Parser, etc.)
│   ├── astgen/
│   │   └── ASTGenerator.py    # Student's AST generator implementation (from
Assignment 2)
│   ├── checker/
│   │   └── StaticChecker.py   # Student's Static Checker implementation (from
Assignment 3)
│   ├── codegen/
│   │   ├── CodeGen.py        # Student's Code Generator class
│   │   ├── Emitter.py        # Student's Emitter helper class
│   │   └── (provided helpers) # JasminCode.py, Frame.py, Io.py, etc.
│   └── runtime/
│       ├── jasmin.jar         # Jasmin assembler (provided)
│       └── io.java            # I/O support class (provided)
├── test/
│   ├── TestUtils.py          # Testing utility class (provided)
│   └── test-codegen.py       # Student's 100 codegen unit tests
└── requirements.txt
```

5. EVALUATION CRITERIA & GRADING

- **Code Generation Correctness (100%):** Grade is based on passing the automated test cases run against your submitted files.
- **Test Cases Penalty:** Writing at least 100 unit tests in `test/test-codegen.py` is mandatory. A penalty of 10% of the total assignment grade will be deducted if `test/test-codegen.py` is missing or contains fewer than 100 test cases.

Test Distribution Table

The 100 tests in `test/test-codegen.py` are grouped into the following categories:

Category	Test Cases	Description
Basic Primitives & Scopes	test_1 - test_20	Primitive types (<code>i32</code> , <code>f32</code> , <code>bool</code> , <code>string</code>), I/O functions (<code>print_i32</code> , <code>print_f32</code> , <code>print_bool</code> , <code>print_string</code> , <code>println</code>), variable assignments, scope shadowing, and implicit coercion from <code>i32</code> to <code>f32</code> .

Composite Data Structures	test_21 - test_40	Struct instantiation & field access/mutation, multidimensional arrays (<code>[[i32; 2]; 3]</code>), heap-allocated <code>Box<T></code> , and pointer dereference assignments.
Expressions & Operations	test_41 - test_54	Arithmetic operators, string & boolean comparisons, short-circuit evaluation (<code>&&</code> , <code> </code>) with side effects, and operator precedence.
Statements & Control Flow	test_55 - test_95	if-else branching, C-style switch with fallthrough & default, while loops, range-based for loops, loop jump controls (<code>break</code> , <code>continue</code>), nested loops, and FizzBuzz.
Complex Algorithms & Recursion	test_96 - test_100	Recursive factorial & fibonacci, mutual recursion (<code>is_even/is_odd</code>), recursive binary search tree with <code>Box<Node></code> , and multi-way tree traversal.

6. DELIVERABLES

Students must submit:

- `src/codegen/CodeGen.py`
- `src/codegen/Emitter.py`
- `test/test-codegen.py`

7. DEADLINE

- **Submission Deadline:** Refer to the [deadlines.json](#) Gist file for the exact due date for Assignment 4.

8. CHANGELOG

- First release under v1.0.